

CNN Accelerator 구현 계획

CNN Accelerator 구현 계획

보드: Arty-A7 100T (DSP48E1 240, BRAM 135, LUT 63K) **CNN**: Conv1(1→8, 3×3) → ReLU → Conv2(8→16, 3×3) → ReLU → MaxPool(2×2) → FC(2304→10)

제공: signed INT8 가중치, 입력 이미지 10K, 기대 logit (.npy 5개)

제출: INT8 기본 가속기 + (선택) lower-bit 가속기 별도. 둘 다 검증 필수.

마감: 작업 6/5까지, 보고서 6/19

원칙: 단계별 동작 베이스라인 확보 후 다음으로. 기본 구현 완료 후 양자화 시도.

사전 확인 (완료)

- 과제 모델 INT8 baseline 정확도 **93%** (Python으로 측정 완료, Claude Code 활용)
- 양자화 변형 정확도 측정은 1단계 기본 구현 완료 후 진행 (교수님 가이드)

병목 분석

Conv1: 1→8 ch, 26×26 output → $8 \times 26^2 \times 9 = 48,672 \text{ MAC/image}$ Conv2: 8→16 ch, 24×24 output → $16 \times 8 \times 24^2 \times 9 = 663,552 \text{ MAC/image}$ ← 병목 FC: 2304 → 10 → **23,040 MAC/image**

Conv2가 전체 MAC의 약 **90%**. Winograd는 Conv2에만 적용해도 큰 효과.

알고리즘 진행 경로

Output Stationary Direct + SIMD → 시스템 최적화 + 오버클럭 → (Python 양자화 정확도 측정) → Conv2를 복소수 F(4,3) Gauss로 교체 → 복소수 F(4,3) + 오버클럭

F(2,3)은 복소수 F(4,3) 대비 throughput 우위가 크지 않고, 중간 검증 단계로의 가치가 작아 생략.

양자화 원칙

- 기본 트랙: 가중치/activation 모두 signed INT8 (과제 명세대로, LSB 10bit 버림 + saturation). baseline 93% 확인됨.
- **Lower-bit** 트랙 (추가 점수, 별도 IP): 전 레이어 균일하게 weight 추가 양자화 (INT7~INT4, Log2). Activation은 INT8 유지 (calibration 위험).
- Winograd는 Conv2에만 적용 (병목 집중). Conv1, FC는 Direct.

SIMD packing 한도 (참고)

DSP48E1: $2N + M \leq 25$ (N = A측 packing 비트, M = B측 단일 비트)

- Direct + INT8 weight + INT8 activation: $24 \leq 25$ ✓

- 복소수 F(4,3) 변환 후 (input m, weight n bit): $V = m+6$, $U = n+4 \rightarrow$ INT8 input/weight면 $2(8+4) + (8+6) = 38$, 한도 크게 초과 $\rightarrow$ **SIMD** 불가
 - $\rightarrow$ 본 가속기 (Arty A7-100T + INT8)에서 복소수 Winograd는 SIMD 없이 큰 비트 곱셈 (1 DSP/mul). 복소수 곱셈은 곱셈 수 적은 **Gauss (38 mul)** 채택.
-

1단계 — Verilog Baseline: Output Stationary Direct + SIMD (~5/24)

목표: 안정 동작 가속기. 명세상 기본 INT8 가속기. 이게 가장 먼저 완성되어야 할 것.

- TA1 골격 재활용 (CSR, AXI BRAM Ctrl, top_memory_ctrl)
- BRAM 배치 (IMEM ping-pong, WMEM, FMAP1, FMAP2, POOL_OUT, RESULT)
- DSP48E1 SIMD INT8 dual-MAC 모듈 단독 검증 후 사용
- **Output Stationary PE** ($a \times w$ 누적 형태로 추상화 $\rightarrow$ 후속 단계에서 재활용)
- Conv1, Conv2 Direct + SIMD (output channel pair 공통 activation)
- MaxPool, FC, ReLU, INT8 saturation (LSB 10bit 버림)
- PL timer (CSR_TIMER_START 트리거, 첫 weight write ~ 마지막 output read)
- Vitis 드라이버 (가중치 $\rightarrow$ 10K 이미지 루프 $\rightarrow$ 결과)
- 클럭 100MHz

검증: 보드 결과가 기대 logit과 비트 매칭 + 정확도 93% 재현 산출: 동작하는 INT8 가속기. 명세상 기본 제출물. 이게 안전 baseline.

5/20-22 Design Review: 발표가 아니라 교수님과 질문/논의하는 자리. 이 시점에서 1단계 진행 상황 + 알고리즘 선택 (복소수 F(4,3) Gauss) 논거 공유.

2단계 — 시스템 최적화 + 오버클럭 (~5/31)

목표: 1단계 위에 시스템 효율 극대화. 알고리즘 변경 없이 latency 감소.

- IMEM/RESULT ping-pong 완성 (PS-PL overlap)
- Conv2 $\rightarrow$ Pool 또는 Pool $\rightarrow$ FC streaming fusion (가능하면)
- **균일 클럭 향상** (100 $\rightarrow$ 150 $\rightarrow$ 200MHz 단계적)
 - DSP48E1 -1 grade 최대 464MHz, BRAM 388MHz 한도. 현실 design은 critical path가 결정
 - 기법: DSP 풀 파이프라인 (A_REG, B_REG, M_REG, P_REG), BRAM output register, critical path register 분할
 - Multi-pumping은 CDC 부담으로 배제, 균일 오버클럭이 단순함과 효과의 균형

검증: 1단계 결과와 비트 일치, Vivado Fmax 확인 산출: 오버클럭된 Direct+SIMD 가속기. 여기까지가 안전 **baseline**. 6/5 final 제출 안전선.

2.5단계 — Python 양자화 정확도 측정 (~6/1, 2단계와 병행 가능)

목적: 양자화 변형의 MNIST 10K 정확도 측정. 1, 2단계 완료 후 / 또는 합성 대기 시간에 병행. 결과에 따라 3단계 (복소수 F(4,3)) + lower-bit IP 진입 결정.

기본 INT8 baseline 정확도는 93%로 이미 확인됨.

측정 방법

- 과제 .npy 5개 로드 (input, output, layer1_0_weight, layer2_0_weight, fc1_weight)
- numpy로 실제 **CNN** 전체 추론 (Conv1 → ReLU → Conv2 → ReLU → MaxPool → FC) 10K 이미지
- 명세대로 INT8 saturation (LSB 10bit 버림 + saturation)
- INT8 Direct가 reference, 기대 logit과 비트 매칭 (이미 확인)

측정 시나리오

속도 트랙 (Conv2만 복소수 F(4,3), 나머지 Direct INT8):

1. 복소수 F(4,3) Gauss + INT8 weight + U/V 큰 비트 (옵션 A, 무손실 기대) — 3단계 진입 정당화 용
2. 복소수 F(4,3) + U → INT8 + V → INT9 양자화 (양자화 손실 측정)
3. 복소수 F(4,3) + U → INT8 + V → INT8 양자화
4. 복소수 F(4,3) + INT4 weight 사전 양자화 (옵션 D)

자원 트랙 (Direct, 전 레이어 균일 weight 양자화): 5. Direct + INT7 weight 6. Direct + INT6 weight 7. Direct + INT5 weight 8. Direct + INT4 weight 9. Direct + signed Log2 4-bit weight 10. Direct + signed Log2 3-bit weight (시간 되면)

각 시나리오 측정:

- 절대 정확도 (MNIST 10K)
- Reference 일치율: $\text{argmax}(\text{my_logit}) == \text{argmax}(\text{expected_logit})$ 비율

양자화는 layer별 scale 재조정 후 round-to-nearest.

산출: 정확도 비교표. 두 결정:

- 시나리오 1이 무손실 (93% 유지) → 3단계 복소수 F(4,3) Gauss Verilog 진입 확정
- 자원 트랙에서 정확도 OK한 양자화 → lower-bit IP 트랙 진입 (별도 점수)

3단계 — Conv2를 복소수 F(4,3) Gauss로 교체 (오버클럭 없이) (~6/3)

목표: Conv2 병목에 알고리즘 교체. 곱셈 수 절감 (Direct 72 → 복소수 F(4,3) Gauss 38/16 $\approx$ 2.375 mul/output pixel). 클럭은 100MHz로 보수적, 알고리즘 변경에만 집중.

- 점 집합 $\{0, \pm 1, \pm i, \infty\}$ 복소수 F(4,3)
- 입력 변환 $B^T d B$ (가산기 트리, 곱셈 0, 변환 후 V 위치별 8~14 bit)
- 가중치 변환 $G g G^T$ (사전 계산 또는 PL 초기화, 변환 후 U 위치별 8~12 bit)
- Element-wise mul: **Gauss** 복소수 곱 (3 real mul/복소수 곱셈, 38 mul/tile)
 - 변환 후 비트 폭이 DSP48E1 SIMD 한도 ($2N+M \leq 25$)를 초과해 SIMD 불가
 - SIMD 못 쓰는 환경에서는 곱셈 수가 적은 Gauss (38) > Naive (40)
 - 큰 비트 그대로 곱셈, INT8 명세 무손실 유지 (변환 행렬 Gaussian integer + $\frac{1}{4}$ 스케일)
- 출력 변환 $A^T M A$ (가산기 트리, $\frac{1}{4}$ 흡수)
- Output Stationary PE 재활용 (1단계와 동일 구조, 데이터 공급만 변경)
- Conv1, FC는 1단계 그대로 (Direct + SIMD)
- 클럭 100MHz 고정

왜 오버클럭 분리: 복소수 데이터패스 (실/허 분리, 변환 가산기 트리, Gauss 5 덧셈 트리) 자체가 큰 변경. 동시에 오버클럭 시도하면 디버깅 지옥. 분리 검증.

검증: 2.5단계 시나리오 1 (복소수 F(4,3) Gauss) Python sim과 비트 일치, 정확도 93% 산출: 복소수 알고리즘 동작 가속기.

4단계 — 복소수 F(4,3) + 오버클럭 (~6/5)

목표: 3단계 + 2단계의 오버클럭 기법 결합. 최종 가속기.

- 3단계 복소수 회로에 pipeline stage 추가
- Critical path 재분석 (복소수 변환 가산기 트리가 새 bottleneck 가능성)
- 150 → 200MHz 단계적 목표

검증: 3단계 결과와 비트 일치 + Fmax 확인 산출: 최종 가속기. 6/5 final 시점에 시연 가능.

Lower-bit 양자화 IP (추가 점수, 병행 가능)

조건: 2.5단계 자원 트랙에서 정확도 OK 확인 + 시간 여유

- 정확도 OK 양자화 선택 (INT4-7 또는 Log2)
- 전 레이어 균일 양자화 (Conv1, Conv2, FC 모두)
- Direct 가속기 별도 IP로 (1단계 구조 재활용, MAC 비트 폭만 축소)
- INT4 SIMD: A에 INT4 packing 더 많이 가능 (DSP 효율 ↑)
- Log2 weight: 곱셈 → 시프트, DSP 사용 감소

명세 요구: 추가 점수 받으려면 INT8 가속기 및 lower-bit 가속기 둘 다 제출 + 검증. 시간 없으면 1순위 (메인 가속기) 완성에 집중.

보고서 (6/5 ~ 6/19)

- Theory: 2D conv, Winograd 유도, 복소수 **F(4,3)** 유도 (영구 노트)
- Architecture: 시스템 다이어그램, Output Stationary 데이터플로우
- Implementation: RTL, CSR, Vitis, 변환 회로
- Results: 단계별 latency (보드 실측), 자원, 정확도
- Discussion: 알고리즘 trade-off, **Gauss vs Naive (Arty A7 SIMD 한도 분석)**, 변환 후 비트 폭, 오버클럭 vs Multi-pumping, Polar/사원수/NTT Future Work
- 제출: TAS2_T#.zip (코드), TAS2_T#_이름_학번.pdf (개인 보고서)

Fallback

단계 실패 시	차선택
1단계	TA1 sobel_ip 구조 최대 재활용. 명세상 기본 INT8 가속기는 무조건 동작해야 함
2단계 (오버클럭 실패)	100MHz 그대로 제출, 복소수로 진행
2.5단계	양자화 정확도 측정 못해도 시나리오 1 (Gauss + INT8) 무손실은 이론적으로 보장
3단계 (복소수 실패)	2단계 (Direct+SIMD 오버클럭) 그대로 제출
4단계 (복소수+오버클럭 실패)	3단계 (복소수 100MHz) 그대로 제출
Lower-bit IP	시간 부족 시 skip. 메인 가속기 완성 우선

각 단계 끝에서 다음 진입 결정. 위험 시 이전 단계 결과로 제출.

핵심: 1단계 (INT8 Direct+SIMD)는 명세 만족 위해 필수. 2단계는 안전 baseline. 3, 4단계는 GOAT 도전.

일정 요약

날짜	단계
완료	INT8 baseline 정확도 측정 (93%, Python)

날짜	단계
~5/24	1단계 Verilog Direct+SIMD @100MHz
5/20~22	Design review (질문/논의 시간, 진행 상황 공유)
~5/31	2단계 시스템 최적화 + 오버클럭
~6/1	2.5단계 Python 양자화 정확도 측정 (2단계와 병행)
~6/3	3단계 Conv2 복소수 F(4,3) Gauss @100MHz
~6/5	4단계 복소수 F(4,3) + 오버클럭
6/5	Final (시연 또는 결과 제출)
~6/19	보고서 제출

자산

- Winograd_Convolution_Notes.md : 이론 (복소수 F(4,3) 유도, 비트 폭 분석, SIMD 호환성)
- Winograd_for_MNIST_Accelerator.pdf : 문제-해결 흐름 정리 (공유용)
- 과제 .npz 5개: input.npz , output.npz , layer1_0_weight.npz , layer2_0_weight.npz , fc1_weight.npz
- TA1 코드: sobel_ip.v , top_memory_ctrlr.v , csr_slave_axi_inner.v
- 참고: UG479 (DSP48E1), WP486 (INT8 packing), DS181 (Artix-7)